Ders 20 Genişletilmiş Notu: İşletim Sistemi Kavramları, Java'da Thread'ler, JUnit, Thread Havuzu ve Senkronizasyon

Ders 20 Genişletilmiş Notu: İşletim Sistemi Kavramları, Java'da Thread'ler, JUnit, Thread Havuzu ve Senkronizasyon

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Program, Proses ve Thread
- 2. İşlemci Komutları Nasıl Çalıştırır?
- 3. Zaman Paylaşımı ve Bağlam Değiştirme
- 4. Multitasking, Multiprocessing ve Multithreading
- 5. Proses Kontrol Bloğu ve Thread Kontrol Bloğu
- 6. Eş Zamanlı Çalışma ve Paralelleştirme
- 7. Thread Durumları ve Doğru Bekleme
- 8. Java'da Thread Oluşturma: Runnable ve Maç Örneği
- 9. Derste Çalıştırılan Thread Örnekleri
- 10. JUnit ile Birim Test
- 11. Thread Havuzu
- 12. run Metodunda Exception
- 13. Yarış Durumu ve Klasik Senkronizasyon Problemleri
- 14. Banka Hesabı Simülasyonu
- 15. Preemptive ve Non-Preemptive Zamanlama
- 16. Alice ve Bob: Kaybolan Güncelleme
- 17. Karşılıklı Dışlama: Lock, Semafor ve Monitör
- 18. synchronized ile Çözüm
- 19. Thread-Safe Veri Yapıları
- 20. Grafik Arayüzden Web'e: MVC'ye İlk Bakış

1. Program, Proses ve Thread

Hoca multithreading'e geçmeden önce, slaytlarda olmayan ama bilinmesi gereken üç kavramı açtı: program, proses (process) ve thread. Bu kavramlar normalde işletim sistemleri dersinin konusudur ve gelecek dönem o derste ayrıntılı görülecektir.

Program, Java ya da C gibi herhangi bir dilde yazılmış, bizim anladığımız ama makinenin anlamadığı komutlar bütünüdür. Bunu bilgisayarın anladığı dile dönüştürmek için derleyicili bir dilse derleriz (compile), yorumlayıcılı bir dilse yorumlarız (interpret). Birden fazla program parçası varsa bunları bağlarız (link). Sonuçta çalıştırılabilir bir dosya (executable file) ortaya çıkar.

Proses, çalışan programdır (running program). Bir dosyaya çift tıkladığımızda ya da komut satırından adını yazdığımızda, yükleyici (loader) çalıştırılabilir programın bir kopyasını ana belleğe yükler. Bellekteki bu örneğin özel adı prosestir. Hoca bunu Görev Yöneticisi'nde (Task

Manager) gösterdi: Details sekmesinde çalışan prosesler görünür ve her birinin bir PID'si vardır. PID, işletim sisteminin prosese verdiği tamsayı kimliktir. İşletim sistemi prosese kaynak tahsis ederken, kaynaklarını keserken ya da ona müdahale ederken prosesi bu kimlik üzerinden yönetir.

Thread, çalışan bir prosesin başlangıç noktasından bitişine doğru giden akışıdır. Programlar `main`'den başlar ve bitiş noktasına kadar ilerler; bu akış bir thread'dir. Slayttaki ifadeyle "each process must have at least one thread": Her prosesin en az bir thread'i olmak zorundadır, çünkü bir program mutlaka bir yerden başlayıp bir yere kadar gidecektir.

Bu üç kavramın bu dersteki yeri şudur: Bir program yazarız, onu proses haline getiririz ve bu prosesteki thread'leri yönetmeye çalışırız. Multithread çalışmada program `main`'den başlar, belli yerlerde tek thread ile giderken belli yerlerde parçalara ayrılır, sonra belli bir noktada parçalar yeniden birleşir ve program tamamlanır. Böylece iş bölünmüş olur.

2. İşlemci Komutları Nasıl Çalıştırır?

İşlemci bir programdaki komutları tek tek çalıştırır. CPU'nun içinde komutların işlendiği aritmetik lojik ünite (ALU) vardır. Hocanın lojik devreler ve bilgisayar organizasyonu derslerine atıfla özetlediği komut döngüsü şöyledir:

1. **Fetch:** Komut getirilir.
2. **Decode:** Komut çözülür.
3. **Execute:** Komut işletilir.
4. **Write back:** Gerekirse sonuç geri yazılır.

Bir birim zamanda normalde bir execution yapılır. Yani tek bir işlem birimi varsa, belli bir T anında yalnızca bir komut çalışır. Hoca burada bir soru sordu: Madem öyle, ben yazı yazarken bir yandan konuşuyorum, bilgisayarda aynı anda bir sürü program çalışıyor; bu nasıl oluyor?

Cevabın iki parçası var:

- **Çok çekirdek:** Günümüz işlemcileri tek çekirdekli (single core) değildir. Çok çekirdekli (multi core) işlemciler bellekten Zoom'un bir komutunu, OneNote'un başka bir komutunu alıp ayrı kollardan aynı anda yürütebilir. Birden fazla işlemci varsa işlem elemanı sayısı daha da artar.
- **Prosesler arası geçiş:** İşletim sistemi, prosesler arasında yönetimli bir geçiş (switching) yapar. Bir sonraki bölümün konusu budur.

3. Zaman Paylaşımı ve Bağlam Değiştirme

$P_1, P_2, P_3, \dots, P_n$ prosesleri olduğunu düşünelim. Hepsi bir market kasası gibi tek bir işlemciyi kullanacak. İşletim sistemi işlemciyi bunlara nasıl kullandırır?

Hoca iki yöntem adı verdi, zaman paylaşımı (time sharing) ve frekans paylaşımı (frequency sharing), ama frekans paylaşımının burada pek söz konusu olmadığını hemen ekledi: İşlemci 4,9 GHz gibi belli bir frekansta çalışır. Elde kalan yöntem zaman paylaşımıdır.

> **Ders sonrası not:** Hoca "temel olarak iki yöntem var" diyerek frekans paylaşımını da saydı, sonra burada söz konusu olmadığını söyledi. Frekans paylaşımı büyük olasılıkla haberleşme kanallarına ait bir kavramdır ve işlemci zamanlamasında bir yöntem değildir. Bu derste geçerli olan zaman paylaşımıdır.

Zaman paylaşımında işletim sistemi, $t = 0$ anından başlayarak belli aralıklarla $(T_1, T_2, T_3, \dots)$ sırayı dağıtır. P_1'e "sana şu kadar zaman verdim, kendi komutlarından bir kısmını çalıştır" der. Süre dolunca P_1 işlemciden çıkar, sıra P_2'ye, sonra P_3'e geçer, sonra yeniden P_1'e gelir. Bu, bilgisayar açıldıktan kapanana kadar sürer.

Hocanın birinci benzetmesi bir oturumdur: Herkes konuşmak ister, ama konuşabilmek için mikrofon gerekir. Mikrofonu elinde tutan, yani izni elinde bulunduran proses çalışabilir.

3.1 Market Kasası Örneği

İkinci benzetme bir markettir: Bir kasa ve birkaç müşteri var. Kasiyer "hiçbirinizi kırmayacağım, hepinizden birer parça alacağım" der ve şöyle ilerler:

Müşteri	Toplam ürün	Bu turda geçen	Kalan
M_1	10 birim	5 birim	$10 - 5 = 5$ birim
M_2	8 birim	4 birim	$8 - 4 = 4$ birim
M_3	7 birim	2 birim	$7 - 2 = 5$ birim (ara adım eklendi)

Son müşteriden (M_n) sonra sıra yeniden M_1'e gelir ve onun kalan 5 birimi üzerinden devam edilir.

Sorun şudur: M_1'den M_2'ye geçerken, M_1 için ne kadar ürün geçirildiği, ne kadar para alınacağı ve hangi ürünlerin kaldığı bir yerde saklanmalıdır. Aynısı M_2, M_3 ve diğerleri için de geçerlidir. Her geçişte ek bilgi saklanır.

3.2 Bağlam Değiştirme ve Kuyruk

Bu ara geçişlerin adı **bağlam değiştirme (context switching)** dir. Bir proses işlemciyi kullanırken işlemciden çıkarılır, beklemeye alınır, diğer proses işlemciye alınır ve işlemlerine devam eder.

Burada veri yapılarından bildiğimiz kuyruk (queue) yapısı vardır. M_1 işlemciden çıkınca kuyruğun sonuna geçer. Prosesler öncelikliyse kuyruk öncelikli kuyruk olur: Daha öncelikli bir proses kuyruğun sonuna değil, ortalarına ya da başına yerleşebilir. Genellikle en yüksek öncelikte tek bir proses olmaz, birden fazla proses olur, dolayısıyla kuyruk mekanizması her durumda işler. Hoca Görev Yöneticisi'nde sütun ekleyerek "Base priority" değerlerini gösterdi: Normal, High, Above normal gibi seviyeler görünür.

4. Multitasking, Multiprocessing ve Multithreading

Multitasking (çok görevlilik), aynı anda birden fazla işin yapılmasıdır. Hoca bunu genel anlamda ikiye ayırdı: multiprocess ve multithread.

4.1 Multiprocessing

Multiprocessing, bir T anında birden fazla prosesin çalıştırılmasıdır. Hoca burada sınıfa sordu.

Soru: Tek işlemci ve tek çekirdekle multiprocess mümkün müdür? **Cevap:** Mümkün değildir. Tek çekirdekte tek bir işlem birimi vardır ve birim zamanda bir komut işlenir. Bir prosesi çalıştırırken başka bir prosesi aynı anda çalıştıramaz. Gerçek multiprocessing için ya çok çekirdekli bir işlemci ya da birden fazla CPU gerekir.

Tek çekirdekte mümkün olan şey **sanal paralellik (virtual parallelism)** dir ve bağlam değiştirme ile sağlanır. İşlemci gigahertz mertebesinde çalıştığı için bir proses girer, nanosaniyeler mertebesinde öbürü girer. Biz bu değişimi algılayamayız ve sanki her şey aynı anda çalışıyormuş gibi görünür. Çok çekirdekte ve çok CPU'da bunu hissetmeyiz bile.

Hocanın tarihsel gözlemi şudur: Windows 3.1 gibi çoklu işlem özelliğinin yeni geldiği eski sistemlerde işlemciler ve bellekler bu kadar gelişmiş değildi. Bağlam değiştirme sırasında, yani bir prosesin hesabı kitabı yapılıp öbürüne geçilirken, gecikme oluşurdu ve bu gecikme diğerlerini de tetiklerdi. Eski bilgisayarlarda programların kısa süre kilitlenip sonra devam etmesinin bir sebebi de buydu.

4.2 Multithreading

Multithreading, bir prosesin kendi içinde alt küçük "prosesçiklere" ayrılmasıdır; bu prosesçiklere thread denir. Prosesin içindeki bazı işlemler alt parçalara bölünür ve bu parçalar eş zamanlı (concurrent) gibi yürütülür.

Hocanın örneği Word'dür. Word'de yazı yazarken yanlış bir şey yazarsanız altı çizilir. Yani yazı yazılırken bir yandan yazım kontrolü, bir yandan tablo ekleme gibi işler yürür. Dikkat: Word'de yazarken müzik dinlemek burada kastedilen şey değildir, o ayrı bir prosestir. Söz konusu olan tek bir prosesin kendi içindeki işleyişidir. Görev Yöneticisi'ndeki thread sütunu da bunu gösterir: Derste bir proseste 26, başka bir proseste 130 thread görüldü.

Hocanın iki kavramı ayırmak için verdiği kısa tanımlar şunlardır:

- **Multithreading:** Bir prosesin kendi içinde alt parçalara bölünmesi.
- **Multiprocessing:** Bir sistemde birden fazla prosesin aynı anda çalışması.

Hocanın vurgusu: Program, proses ve thread'in ne olduğunu; context switching'i; fetch, decode, execute aşamalarını; multitasking, multiprocessing ve multithreading'i ve son ikisinin farkını bilin.

5. Proses Kontrol Bloğu ve Thread Kontrol Bloğu

Bir program belleğe yüklenip proses haline gelince ona bellekte bir yer ayrılır. Burada programın değişkenleri (static ve static olmayan, ilk değer verilmiş ve verilmemiş), `malloc` ile oluşturulan dinamik değişkenler ve programın kodu bulunur. İşlemci bayrakları (flag) da saklanır, çünkü bir fonksiyon çağrısında, özyinelemeli (recursive) ya da normal, işlemcinin durumunun kaydedilmesi gerekir. Assembly dersinde bu konuşulmuştu.

Hoca bir de **proses kontrol bloğu (PCB)** şekli gösterdi. Burada programın içeriği dışında şu bilgiler vardır:

- **Program sayacı (program counter):** İşlemcideki sayacın değeri.
- **Proses durumu (state):** Proses bekliyor mu, çalışıyor mu, sonlandı mı? Prosesler işlemciden çıktıklarında tamamlanmadıysa beklerler; bu bilginin saklanması gerekir.
- **Proses kimliği (PID).**
- **Ebeveyn ve çocuk proses bilgileri:** Bir prosesten başka bir proses oluşturulabilir. İşletim sistemleri dersinde `fork` ve `exec` ile görülecek; ilgili işaretçilerin (pointer) saklanması gerekir.

Hoca, bu dersin kapsamında önemli olanın proses durumu olduğunu söyledi.

Ders sonrası not: Hoca değişkenler, kod ve dinamik bellek dahil bütün bilgilerin "process control bloğu içerisinde" yönetildiğini söyledi. Doğrusu büyük olasılıkla şudur: Kod, değişkenler ve dinamik bellek prosesin bellek alanındadır. PCB ise işletim sisteminin durum, PID ve program sayacı gibi yönetim bilgilerini tuttuğu ayrı bir yapıdır.

5.1 Thread Kontrol Bloğu

Bir prosesin altında birden fazla thread çalışabilir. Bu thread'ler prosese ayrılan ortak alanlara, örneğin global değişkenlere, birlikte erişebilir. Bunun yanında her thread için **thread kontrol bloğu (TCB)** vardır. Yapısı PCB'ye benzer; onun bir alt kümesi gibi düşünülebilir. Prosesle ilgili genel bilgiler PCB'de, her thread'e ait bilgiler o thread'in TCB'sinde durur. Her thread'in kendi yerel değişkenleri olabilir. Thread'ler arasında da geçiş yapılır: Bir thread biraz çalışır, sıra diğerine geçer, sonra geri dönülür. Bu nedenle her thread'in nereden devam edeceğinin saklanması gerekir.

Hoca tek thread'li ve çok thread'li proses şekillerini gösterdi: Çok thread'li proseste ortak bir alan ve her thread'e ait ayrı bir alan bulunur.

Hocanın sezgi için önerdiği deney şudur: Sol elinizi bir yönde çevirirken sağ elinizi tam ters yönde çevirebilir misiniz? Bunu yapabilmek için beynin iki tarafının aynı anda çalışması gerekir. Thread'ler için de benzer bir durum vardır: Normalde bir prosesin altında belli bir anda bir thread çalışır ve aralarında çok hızlı geçiş yapılır. Çok çekirdekli işlemcide ise thread'ler gerçekten paralel çalışabilir.

5.2 Thread'ler Birbirine Müdahale Etmemeli

Paralel çalışan thread'lerin uyumlu (senkron) çalışması gerekir. T_1'in uğraştığı bir veri varsa T_2 ona müdahale etmemelidir. Hocanın Word örneğinde bir thread yazıda değişiklik yapar, öteki yalnızca kontrol eder, değiştirmez; bu ikisi birlikte çalışabilir. Thread'ler birbirinin alanına karışmamalı, ortak iş gerekiyorsa üstteki ortak alanlar üzerinden yapılmalıdır.

> **Hocanın vurgusu:** Ödevlerde "global değişken kullanmayın" dememizin gerekçelerinden biri budur. Bir global değişkeni bir thread artırırken öbürü azaltıyorsa yanlış sonuçlar alınır ve bunun daha ciddi sıkıntıları vardır.

6. Eş Zamanlı Çalışma ve Paralelleştirme

Slaytta multitasking, aynı zamanda birden fazla programın, yani prosesin çalışması olarak geçer: "programs working at the same time can be defined as multiprocessing". Günümüz bilgisayarlarında birden fazla çekirdek vardır. Her çekirdek işlemci mimarisine göre bir ya da daha fazla görev yürütebilir. Slayta göre bir proses, paralel çalışabilecek thread'lere bölünebilir.

Concurrent kelimesi çok karşımıza çıkar ve eş zamanlı demektir. Yeterli donanım kaynağı, yani çekirdek varsa, çalışma süresi ciddi biçimde düşebilir. Burada belirleyici olan işlemcinin gigahertz değerinin yüksekliği değil, komutu işleyecek işlem biriminin bulunmasıdır.

Hocanın örneği iki dizinin eleman eleman toplanmasıdır: $[1, 2, 3]$ ve $[4, 5, 6]$.

- Birinci eleman: $1 + 4 = 5$ (ara adım eklendi)
- İkinci eleman: $2 + 5 = 7$ (ara adım eklendi)
- Üçüncü eleman: $3 + 6 = 9$ (ara adım eklendi)

Sonuç $[5, 7, 9]$ olur. Birinci toplamı bitirip sonra ikinciye geçmenin bir gereği yoktur, çünkü bu toplamlar birbirini etkilemez. $1+4$ yapılırken bir yandan $2+5$, öbür yandan $3+6$ yaptırılabilir. Çok büyük vektörlerde bunu tek bir `for` döngüsüyle yapmak yerine böyle bölmek süreyi kısaltır. Aynı şey matris toplama ve matris çarpımı için de geçerlidir.

İki uyarı vardır:

- **Paralelleştirmenin maliyeti:** Bir proses ya da thread oluşturmak ek zaman alır. Paralel yapayım derken klasik toplamadan daha uzun süren bir sonuç ortaya çıkmamalıdır.
- **Bölme mantıklı olmalı:** Eş zamanlı çalışan thread'lerin bir şekilde senkronize edilmesi gerekir. "Parçalara böldüm, her şey çok güzel çalışacak" diye bir mantık yoktur. Parçaların birbirini ne kadar etkileyeceğine bakılmalıdır.

Hoca, geçen dönem yapısal programlamada C'de thread oluşturmayı gördüklerini, burada ise Java'daki daha kolay kullanımlara bakacaklarını ve işletim sistemi dersi kadar ayrıntıya girmeyeceklerini belirtti.

7. Thread Durumları ve Doğru Bekleme

Hoca durumları proses üzerinden anlattı, aynı durumlar thread'ler için de geçerlidir. Slayttaki durum şeması sözlü olarak şöyle ilerler:

1. **New (yeni):** Oluşturulduktan hemen sonraki durum.
2. **Runnable (çalıştırılabilir):** Planlayıcının kuyruğunda sıra bekler.
3. **Running (çalışıyor):** İşletim sistemi işlemciye göndermiştir, komutları işletilir. İşletim sisteminin altındaki planlayıcı (scheduler) kimin ne kadar çalışacağını belirler.
4. **Dead (ölü):** Çalışma biter, sonlanır (terminate) ya da hata nedeniyle kesilir.
5. **Blocked (bloklanmış):** Çalışan thread bir şeyi beklemek için buraya geçer. Sebep biter bitmez doğrudan işlemciye dönmez, yeniden Runnable olup planlayıcının kuyruğuna girer.

Blocked durumuna geçme sebepleri ve dönüşleri şunlardır:

Sebep	Ne olur?	Nasıl döner?
I/O	Örneğin kullanıcıdan sayı bekleniyor. Boşuna bekletmemek için işlemciden çıkar, I/O servisine alınır.	Kullanıcı sayıyı girip Enter'a basar, I/O tamamlanır.
`wait`	Başka bir amaçla bekler.	`notify` ile bir thread, `notifyAll` ile bütün thread'ler uyandırılır.
`sleep`	İşlemciden çıkar, varlığını gösterecek kadar az kaynak kullanır.	Uyku süresi dolar ya da `interrupt` gelir. Uyandırma her zaman sorunsuz olmaz, exception da oluşabilir.
`join`	Başka bir thread'in tamamlanmasını bekler.	Beklenen thread biter.

Hoca `join` için şu resmi çizdi: Bir thread bir noktaya gelmiş, diğerinin gelmesini bekliyor. Çünkü öteki muhtemelen bir işi tamamlayacak ve belki sonucunu ona devredecek. Hoca I/O ve `wait` sırasında sistem kaynaklarının kısmen kullanıldığını, `sleep` sırasında ise çok az kullanıldığını söyledi.

Planlayıcının yaptığı şey zaman dilimleme (time slicing) ya da `yield` ile sıra dağıtmaktır. Bu, hocanın deyimiyle tamamen işletim sisteminin "paşa gönlüne" göre yapılır. Öncelikler bu paylaşımda kullanılan kriterlerden biridir.

7.1 Thread Nasıl Beklemeli?

Thread aktif değilse önünde bir engel vardır: Ağdan gelecek bir veri, başka bir thread'den gelecek veri, kullanıcıdan alınacak değer ya da programcının koyduğu bir engel. Thread bu engel kalkana kadar beklemelidir. Hocanın benzetmesi, bir yere gideceğiniz arkadaşınızı beklemektir: "Hadi gel" diye beklerken zamanınız tükenir. İşletim sistemleri dersinde bu ilişkiler graflarla gösterilecektir.

> **Hocanın vurgusu:** Busy waiting yapmayın. Bellekte boş komut sıralatmayın, örneğin on bin kez dönen boş bir `for` yazmayın. Thread'i bloklanmış duruma alın, `sleep` ya da `wait` kullanın. Boş döngü amaçsızca kaynak tüketir, uyuyan thread tüketmez.
>
> **Ders sonrası not:** Hoca, uyuyan thread'in uyanmama ihtimali olduğunu ve bunun için "uninterrupted exception" yakalanabileceğini söyledi. Doğru ad `InterruptedException`'dır. Bu exception büyük olasılıkla "uyanamama" durumu için değil, uyuyan thread başka bir thread tarafından kesildiğinde (`interrupt`) fırlatılır. Checked olduğu javac ile denendi: `try-catch` olmadan yazılan `Thread.sleep(500)` satırı `unreported exception InterruptedException; must be caught or declared to be thrown` hatası verir.

8. Java'da Thread Oluşturma: Runnable ve Maç Örneği

Multithreading, çalışan bir görevi ayrı thread'lere bölmektir. Java'da bu iş `Runnable` arayüzü üzerinden yapılır. Slayttaki cümle şudur: "Place the code for the task into the run method of class that implements the Runnable interface." Adımlar şöyledir:

1. Bir sınıf yazılır ve `Runnable` arayüzünü `implements` eder.
2. Thread'in yapacağı iş `run` metoduna yazılır. Word örneğinde bir thread yazı yazar, bir thread yazım kontrolü yapar; her biri için bir sınıf yazılabilir.
3. Bu sınıftan bir nesne oluşturulur.
4. Bu nesne verilerek bir `Thread` nesnesi oluşturulur.
5. Thread'in `start` metodu çağrılır.

> **Hocanın vurgusu:** Thread'i `run` ile değil `start` ile başlatın. `start` denince `run` otomatik çalışır. `Thread` sınıfından kalıtım almak yerine `Runnable`'ı `implements` edin; gereksiz kalıtım fazladan yük getirir.

8.1 Senaryo: Maç İzleyen İnsanlar

Slayttaki örnek "people watching a match" adını taşır. Maç izleyen herkes takımını desteklemek için bağırabilir. Herkes aynı anda heyecanlanmaz, yanınızdakine baktığınızda farklı anlarda bağırdığını görürsünüz; herkes kendi başına buyruktur. Bir kişi 0 ile 1 saniye arasında rastgele bir sürede heyecanlanır. On kez bağırdıktan sonra yorulur ve dinlenir.

`run` metodu parametre almaz ve `void` döndürür. Hoca sınıfta `steps` ve `delay` adlı iki değişken, kurucuda takım ve bağırılacak söz olduğunu anlattı. `sleep` exception üretebileceği için `try` kullanılır. Aşağıdaki kod hocanın sözlü anlatımından kuruldu, javac ile denendi ve çalıştırıldı.

```
public class SportsFan implements Runnable {
    private final String team;
    private final String slogan;
    private final int steps = 10;

    public SportsFan(String team, String slogan) {
        this.team = team;
        this.slogan = slogan;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < steps; i++) {
                System.out.println(team + ": " + slogan);
                int delay = (int) (Math.random() * 1000);
                Thread.sleep(delay);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

> **Ders sonrası not:** Hoca "Java'da da aynıydı, bir metot vardı void döndürüyordu, istersek void pointer alıyorduk" dedi. Burada kastettiği büyük olasılıkla geçen dönem gördükleri C'deki thread fonksiyonudur. Java'da `void` işaretçi yoktur; `run` parametre almaz ve `void` döndürür.

Test sınıfında dört taraftar için dört thread başlatılır. Hocanın slaytı, aynı `Thread` referansını yeniden kullanıyor.

```
public class Match1 {
    public static void main(String[] args) {
        Thread t = new Thread(new SportsFan("GS", "GS rules"));
        t.start();
        t = new Thread(new SportsFan("GS", "GS is the champion"));
        t.start();
        t = new Thread(new SportsFan("FB", "FB rules"));
        t.start();
        t = new Thread(new SportsFan("FB", "FB is the best"));
        t.start();
    }
}
```

`start` çağrısından sonra thread, izin verildiğinde `run` metodunu çalıştırmaya başlar. Örneğin `i = 3`'e kadar çalışır, sıra başkasına geçer, yeniden sıra geldiğinde kaldığı yerden devam eder. Bir adım çalışıp uykuya da geçebilir; o sırada diğer thread'e sıra verilir. Bu tamamen farklı senaryolara açıktır.

Thread ne zaman biter? `run` metodu bitince. Her thread on kez döner, her dönüşte rastgele uyur, döngüden çıkınca işini tamamlamış olur. Program çalıştırıldığında 4 thread 10'ar satır, toplam 40 satır yazar; bu javac ile denendi.

9. Derste Çalıştırılan Thread Örnekleri

Hoca hazır örnekleri NetBeans'te tek tek çalıştırdı.

9.1 Çıktı Sırası Belli Değildir

Birinci maç örneği çalıştırıldığında hangi thread'in önce bitireceği belli değildi. Kodda sonra başlatılan "FB is the best" bir çalıştırmada en sonda, bir sonraki çalıştırmada kendinden önce başlatılanlara sıra gelmeden en başta göründü. Her çalıştırmada sonuç, o anda işletime hangi thread'in alındığına göre değişir.

9.2 start Yerine run Çağırmak

İkinci örnekte `start` yerine doğrudan `t.run()` çağrıldı. Bu durumda önce birinci iş tamamlanır, sonra ikincisi başlar. Bu, normal bir metot çağrısıdır ve yeni thread açılmaz; çalışma sıralı (sequential) olur. Üçüncü örnekte `run`, `Thread` nesnesi yerine doğrudan sınıfın nesnesi üzerinden çağrıldı; sonuç yine aynı sıralı çalışmadır.

```
public class Match2 {
    public static void main(String[] args) {
        Thread t = new Thread(new SportsFan("GS", "GS is the champion"));
        t.run();
        t = new Thread(new SportsFan("FB", "FB is the best"));
        t.run();
        new SportsFan("GS", "GS rules").run();
    }
}
```

Bu kod javac ile denendi. Her söz kendi 10 satırını peş peşe yazdı ve hiç karışma olmadı.

Dördüncü örnekte her thread için ayrı referans oluşturulmuştur (`t1`, `t2`, ...). Bu da multithread çalışmadır. Tek referansla ya da birden fazla referansla başlatmak, çalışma açısından fark yaratmaz. Hoca bu örnekte, prosesin bitip bitmediğinin NetBeans'teki işaretçiden görüldüğünü de gösterdi; yazdırma bittikten kısa süre sonra proses tamamlandı.

9.3 Aslında İki Thread Var

Hoca eski uygulama örneklerine de baktı. Birinde `Thread` sınıfından kalıtım alan bir sınıf vardı. Bunun yerine `Runnable`'ı `implements` etmek daha makuldür, geri kalanı aynıdır. Hoca bu örnekte önemli bir noktayı vurguladı: Kodda tek thread oluşturulmuş gibi görünür ama aslında iki thread vardır. Biri `main`'in kendi çalışması için gereken thread, öbürü bizim oluşturduğumuz thread.

9.4 İki İşçi

Sıradaki örnekte iki işçi nesnesi vardır. Her işçi hangi thread olduğunu ve 0'dan 4'e kadar kaçıncı adımda olduğunu yazar. Thread'siz, yani `run` ile çalıştırılınca önce birinci işçinin 0, 1, 2, 3, 4 adımları, sonra ikincininkiler düzgünce görünür. Hoca bunu önce gösterdi ve gerçek hayatta "önce biri çalışır, sonra öbürü" diye bir şey olmadığını söyledi. `start` ile çalıştırınca önce `Thread-1`'in, sonra `Thread-0`'ın bittiği, adımların az karıştığı ya da biraz daha karıştığı çıktılar görüldü.

Karışmayı artırmak için hoca döngüye `Thread.sleep` ekledi. NetBeans kodu derletmedi: `sleep` için ya `try-catch` kullanılmalı ya da metoda `throws` yazılmalıdır. `run` metodunda `try-catch` kullanıldı. Önce 500, sonra 100 ms ile denendi. Thread beklediği için diğerine sıra gelmesi daha olası hale geldi.

```
public class Worker implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(Thread.currentThread().getName() + " " + i);
```

```
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }

    public static void main(String[] args) {
        Thread worker1 = new Thread(new Worker());
        Thread worker2 = new Thread(new Worker());
        worker1.start();
        worker2.start();
    }
}
```

Bu kod javac ile denendi. Bir çalıştırmanın çıktısı `Thread-1 0`, `Thread-0 0`, `Thread-1 1`, `Thread-0 1` diye sırayla başladı; 100 ms'lik uyku adımları dönüşümlü hale getirdi. Sıra yine garanti değildir. Hoca bunun sebebini de işletim sisteminin asıl amacına bağladı: Kaynakların, yani işlemci ve belleğin etkin kullanımı. Uyuyan thread işlemciyi bıraktığında işletim sistemi işlemciyi boş bırakmaz, başka bir thread'e verir.

9.5 join

Thread'ler birbirini bekleyebilir. `join` ile bir thread, başka bir thread'in bitmesini bekler. Hoca örneğinde `main`'in de kendisinin bir thread olduğunu hatırlattı: `t1.join()` çağrısı, `t1` tamamlanmadan `main`'in sonraki satırlara geçmesine izin vermez. Bu yüzden `t2` ve `t3` ancak `t1` bittikten sonra başlar.

```
class Task implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println(Thread.currentThread().getName() + " " + i);
        }
    }
}

public class JoinDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(new Task());
        Thread t2 = new Thread(new Task());
        Thread t3 = new Thread(new Task());

        t1.start();
        t1.join();      // main, t1 bitene kadar bu satırda bekler

        t2.start();
        t3.start();
    }
}
```

Bu kod javac ile denendi. `Thread-0`'ın beş satırı her zaman önce geldi, ardından `Thread-1` ve `Thread-2` satırları birbirine karıştı. İsimlerdeki 0, 1, 2 sayıları Java'nın thread'lere kendiliğinden verdiği numaralardır.

Hocanın ikinci `join` örneğinde bekleme süresi veriliyordu: `t1.join(süre)`. Bu süre dolunca, `t1` bitmemiş olsa bile diğerlerine sıra geçer ve hepsi çalışabilir hale gelir. Hoca bunun çok kritik olmadığını ama `join`'i bilmenin iyi olacağını söyledi. Aşağıdaki örnek bu anlatımdan kuruldu ve iki thread'e `setName` ile ad veriyor. Derste bir örnekte de `setName` ve `getName` ile thread adları değiştirilip okundu.

```java
class SlowTask implements Runnable {
    @Override
    public void run() {
        try {
            for (int i = 0; i < 5; i++) {
                System.out.println(Thread.currentThread().getName() + " " + i);
                Thread.sleep(500);
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

public class TimedJoin {
    public static void main(String[] args) throws InterruptedException {
        Thread t1 = new Thread(new SlowTask());
        t1.setName("t1");
        Thread t2 = new Thread(new SlowTask());
        t2.setName("t2");

        t1.start();
        t1.join(1000);  // en çok 1000 ms bekler, sonra main devam eder
        t2.start();
    }
}
```

Bu kod javac ile denendi. Çıktı `t1 0`, `t1 1`, `t2 0`, `t1 2`, `t2 1`, ... diye ilerledi. Yani `t2`, `t1` bitmeden başladı.

9.6 Öncelik

Thread'lere öncelik verilebilir. Minimum öncelikli thread'e sıra daha geç, maksimum öncelikli olana daha sık gelir. Proseslerde de aynı şey yapılır; hoca Görev Yöneticisi'nde önceliği değiştirmeyi ve çok etkileşimli işler için "real time" gibi yüksek değerler verilebildiğini de andı.

Derste iki thread çalıştırıldı. Önce minimum öncelikli olan başlatıldı, sonra maksimum öncelikli olan. Çıktıda önce önceliği 10 olan thread görüldü. Hoca, düşük öncelikli olanın önce girdiği zamanların da olabileceğini, bunun sorun olmadığını söyledi. Peş peşe çalıştıkları için beklenen, yüksek öncelikli olanın daha çok çalışması ve daha erken bitmesidir.

```java
class PrintPriority implements Runnable {
    @Override
    public void run() {
        Thread me = Thread.currentThread();
        System.out.println(me.getName() + " priority = " + me.getPriority());
    }
}

public class PriorityDemo {
    public static void main(String[] args) {
        Thread low = new Thread(new PrintPriority());
        Thread high = new Thread(new PrintPriority());
        low.setPriority(Thread.MIN_PRIORITY);
        high.setPriority(Thread.MAX_PRIORITY);
        low.start();
        high.start();
    }
}
```

Bu kod javac ile denendi. Değerler 1 ve 10 olarak yazıldı. Altı çalıştırmanın dördünde `Thread-0` (öncelik 1) önce yazdı. Yani öncelik sırayı garanti etmedi, hocanın uyarısıyla uyumlu.

9.7 Thread Sınıfından Kalıtım ve Anonim Sınıf

Hoca iki alternatif gösterimi daha çalıştırdı. Birincisinde iki sınıf `Thread`'den `extends` alıyor. Biri yazım kontrolünü, öbürü yazı yazmayı temsil edebilir. Nesneler üretilip başlatılınca toplam 3 thread çalışır: `main`, `Simple` ve `Simple2`.

```
class Simple extends Thread {
    @Override
    public void run() {
        System.out.println("task 1");
    }
}

class Simple2 extends Thread {
    @Override
    public void run() {
        System.out.println("task 2");
    }
}

public class MultiTask {
    public static void main(String[] args) {
        new Simple().start();
        new Simple2().start();
    }
}
```

İkincisinde inner class konusunda gördüğümüz anonim sınıf kullanılır: `new Thread()` yazıp bir blok açılır ve `run` burada tanımlanır.

```
public class AnonymousDemo {
    public static void main(String[] args) {
        Thread t1 = new Thread() {
            @Override
            public void run() {
                System.out.println("task 1");
            }
        };
        Thread t2 = new Thread() {
            @Override
            public void run() {
                System.out.println("task 2");
            }
        };
        t1.start();
        t2.start();
    }
}
```

İki kod da javac ile denendi. Derste de bazen önce task 1, bazen önce task 2 görüldü. Hoca bunların güzel yöntemler olmadığını, programı çok şişirdiğini, `Runnable` ile yazmanın daha makul olduğunu söyledi.

10. JUnit ile Birim Test

Hoca thread havuzundan önce, projede lazım olacağı için JUnit'e geçti. Ders slaytlarının 215. sayfasına kadar olan kısım daha önce bitmişti.

10.1 Neden Birim Test?

Büyük bir projede bir sürü sınıf ve metot yazarız. Bunları genellikle `main` üzerinden, çıktıya gözle bakarak test ederiz. Laboratuvarlarda da test sınıfı verilip "çıktı şöyle olacak" denir.

Hocanın örneği bir kitap sınıfıdır: Kitapları listeleyen metodu çağırırız ve belli bir sırayla listelenip listelenmediğine bakarız. Sonuçları tek tek kontrol etmek çok uzun bir iştir.

Birim test (unit test), bir kod biriminin, yani bir sınıfın testini otomatik ve daha formal hale getirmektir. Testin tasarlanması, yürütülmesi ve sonuçların analiz edilmesi (design, execute, analyze) bir araçla yapılırsa çok kolaylaşır. Sonuç olarak yüzde kaçının başarılı olduğunu da görürüz. Yazılım mühendisliği dersinde konunun ayrıntıları görülecektir.

> **Hocanın vurgusu:** Piyasada bu konuda talep var; birim test yapmayı bilmeyenler olduğu için bu derse eklendi. Projenizde kullanmanızı bekliyoruz, test case'lerinizi hazırlayın.

10.2 JUnit Yapısı: Sürüm 3 ve Sürüm 4

JUnit, Java'da bu iş için kullanılan bir araçtır. Çerçevenin içinde test durumu (test case) oluşturulur ve `Assert` sınıfındaki metotlarla eşittir, eşit değildir, doğru, yanlış, null, null değil gibi kontroller yapılır. Bir metottan beklediğimiz sonucu alıp almadığımızı böyle test ederiz.

Özellik	JUnit 3.x	JUnit 4.x
Test sınıfı	`TestCase`'ten kalıtım alır	Kalıtım gerekmez
Test metodu adı	`testXXX` biçiminde başlamalı	`test` ile başlaması gerekmez
Metodu test olarak işaretleme	Addan anlaşılır	Önüne `@Test` anotasyonu yazılır
Ön hazırlık	`setUp` her test metodundan önce çalışır	Slaytta ayrıca anılmadı
Testleri birleştirme	Statik `suite` metodu	Slaytta ayrıca anılmadı
Exception	Slaytta ayrıca anılmadı	Oluşan exception'ı bildiren destek vardır

Her iki sürümde de test durumları ayrı sınıflarda yazılır.

10.3 Fowler'ın Kiralama Örneği

Hoca, Fowler'ın kitabındaki paketten alınmış bir örnek üzerinden ilerledi: `Movie`, `Rental` ve `Customer` sınıfları. Bir müşteri bir kiralama işlemi yapar ve birden fazla film kiralayabilir. Sınıflar arasındaki ilişki association'dır; şekil aggregation gibi görünse de hoca bunun association olduğunu belirtti.

Kullanıcı `statement` metodunu çağırdığında şu silsile işler:

1. Bütün kiralamalar dolaşılır.
2. Her kiralamanın filminden `getTitle` ile film adı alınır.
3. `getCharge` ile ücret alınır.
4. Sonuçlar birleştirilip döndürülür.

Bunların doğru olup olmadığını öğrenmek için slaytta JUnit 3 biçiminde `TestCustomer extends TestCase` sınıfı yazılıyor. Sınıfta `Customer` ve `Movie` referansları vardır. Aşağıdaki iskelet anlatımdan kuruldu ve JUnit sınıfları için yazılan küçük saplamalarla javac ile denendi. Makinede JUnit kütüphanesi olmadığından testler gerçek JUnit ile çalıştırılmadı.

```
import junit.framework.TestCase;

public class TestCustomer extends TestCase {
    private Customer yunus;

    @Override
    protected void setUp() {
        yunus = new Customer("Yunus");
    }

    public void testGetName() {
```

```
        assertEquals("Yunus", yunus.getName());
    }

    public void testStatementForNoRentals() {
        String expected = "Rental Record for Yunus\n"; // devamı slaytta
        assertEquals(expected, yunus.statement());
    }
}
```

`getName` bir `String` döndürür. `assertEquals`, beklenen değeri metottan dönen sonuçla karşılaştırır. Mantık şudur: Oluşturduğunuz test durumlarının beklenen sonuçlarını buraya yazarsınız, elde edilen sonuçlarla eşitliğine ya da diğer durumlarına bakarsınız. Slaytta uzun süreli kiralamalar için de bir test vardır: Film nesneleri müşteriye verilir, metot bir kez çağrılır ve "bu metottan böyle bir şey dönmesini bekleriz" diye beklenen metin yazılır. Yani metodun nasıl bir sonuç döndüreceğini önceden bilmemiz gerekir.

> **Ders sonrası not:** Hoca, assert metotlarının çoğunun boolean ya da nesne döndürdüğünü ve karşılaştırma sonrası "true ya da false dönebileceğini" söyledi. Doğrusu büyük olasılıkla şudur: Assert metotları değer döndürmez, koşul sağlanmazsa testi başarısız sayan bir hata fırlatır. JUnit bu hataları toplayarak sonucu raporlar.

Bütün testleri birlikte çalıştırmak için slaytta bir `suite` metodu vardır. Statiktir ve `Test` türünde bir şey döndürür:

```
import junit.framework.Test;
import junit.framework.TestSuite;

public class TestAll {
    public static Test suite() {
        TestSuite suite = new TestSuite();
        suite.addTestSuite(TestCustomer.class);
        suite.addTestSuite(TestMovie.class);
        return suite;
    }
}
```

Slaytta `TestAll`'a sağ tıklayıp "Run as JUnit Test" denir. Sonuç, herhangi bir hata olup olmadığını ve testlerin yüzde kaçının başarılı olduğunu gösterir. Yüzde 100 ise oradaki bütün metotlar test edilen durumlar için doğru çalışıyor demektir.

> **Ek bilgi (derste anlatılmadı):** Aynı test JUnit 4 biçiminde `TestCase`'ten kalıtım almadan, büyük olasılıkla şöyle yazılır: `@Test public void getNameTest() { assertEquals("Yunus", new Customer("Yunus").getName()); }`. Bunun için `org.junit.Test` ve `org.junit.Assert.assertEquals` statik olarak içe aktarılır. Metot adının `test` ile başlaması gerekmez.

10.4 NetBeans'te JUnit

Hoca NetBeans'te "Test Project" seçeneğini gösterdi. Test sınıfı yazılmadığı için herhangi bir testin çalıştırılmadığını bildirdi. Bir öğrencinin sohbete yazdığı gibi, sınıfa tıklayıp "test file" seçilince o sınıfın testi açılıyor. Hoca, burada da slayttakine benzer bir arayüz olduğunu söyledi. Hoca, örneğin kodunu kendi klasörlerinde bulamadığı için çalıştırıp gösteremedi.

11. Thread Havuzu

Bir programı 10 ya da 20 thread'e bölmek isteyebiliriz, ya da en verimli çalışacak dinamik bir sayı isteyebiliriz. Bu sayıyı bulmak zordur. Az sayıda thread için her birini elle oluşturmak kabul edilebilir. Fakat çok sayıda görev varsa thread oluşturmak ayrı bir maliyettir ve her biri için kodu yazıp `new` demek de uzundur. Bu nedenle çok görev çalıştıracaksak thread havuzu (thread pool) kullanmak daha makuldür. Üç beş thread'i alt alta kodlayıp ayrı ayrı çalıştırmak da iyi bir yol değildir.

Ders sonrası not: Hoca, gerçek dünyada her thread'in "kendine has bir CPU tarafından yönetilmesi gerektiğini" söyledi. Bu, dersin başındaki anlatımla uyuşmaz: Thread'ler çekirdekler üzerinde zaman paylaşımıyla çalışır, her birine ayrı CPU gerekmez. Burada kastedilen büyük olasılıkla her thread'in işletim sistemine bir yönetim maliyeti getirmesidir.

11.1 Arayüzler ve Metotlar

Bu iş için `java.util.concurrent` paketindeki türler kullanılır.

Tür	Metot	Görevi
`Executor`	`execute(Runnable)`	Verilen görevi çalıştırır.
`ExecutorService`	`shutdown()`	Güvenli kapatma: Yeni görev kabul etmez, çalışanların bitmesine izin verir.
`ExecutorService`	`shutdownNow()`	Derhal kapatmayı dener ve bir liste döndürür.
`ExecutorService`	`isShutdown()`	Kapatma başlatılmış mı?
`ExecutorService`	`isTerminated()`	Havuzdaki bütün görevler bitti mi? `main` içinde beklerken kullanılabilir.
`Executors`	`newFixedThreadPool(n)`	n thread'lik havuz oluşturur; 5 verilirse 5 thread'lik havuz olur.
`Executors`	`newCachedThreadPool()`	İhtiyaç kadar thread oluşturur, daha önce oluşturulmuş boştaki thread'leri yeniden kullanır.

Ders sonrası not: Hoca, `shutdownNow`'ın tamamlanmamış thread'leri kapattığını ve tamamlanmamışların listesini döndürdüğünü söyledi. Doğrusu, çalışan görevleri kesmeye çalışır ve hiç başlamamış görevlerin listesini döndürür. Bu javac ile denendi: Tek thread'lik havuza üç görev verildi, `shutdownNow` sonrasında çalışan görev `InterruptedException` aldı ve dönen listenin boyu 2 oldu. Yani liste, çalışırken kesilen görevi içermedi.

Hocanın `shutdown` için söylediği de javac ile denendi: Kapatıldıktan sonra `execute` çağrısı `RejectedExecutionException` verdi.

11.2 Havuzlu Maç Örneği

`SportsFan` sınıfında hiçbir değişiklik yoktur. Her biri için ayrı thread oluşturmak yerine bir `ExecutorService` nesnesi üretilir, görev nesneleri havuza gönderilir, sonra `shutdown` denir.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

class SportsFan implements Runnable {
    private final String slogan;

    SportsFan(String slogan) {
        this.slogan = slogan;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                System.out.println(slogan);
                Thread.sleep((int) (Math.random() * 1000));
```

```
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}

public class MatchWithPool {
    public static void main(String[] args) {
        long start = System.currentTimeMillis();
        ExecutorService pool = Executors.newCachedThreadPool();
        pool.execute(new SportsFan("GS rules"));
        pool.execute(new SportsFan("GS is the champion"));
        pool.execute(new SportsFan("FB rules"));
        pool.execute(new SportsFan("FB is the best"));
        pool.shutdown();
        System.out.println("Shutdown initiated");
        while (!pool.isTerminated()) {
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        System.out.println("Shutdown complete, ms: " + (System.currentTimeMillis() - start));
    }
}
```

Derste ve javac ile yapılan denemede aynı davranış görüldü: “Shutdown initiated” yazısı çıkar, ama diğer thread’ler hâlâ yazmaya devam eder. Hepsi bitince “Shutdown complete” ve geçen süre yazılır. Denemede ilk satırlardan sonra “Shutdown initiated” geldi, toplam 42 satır oluştu. Yani `shutdown` çağrısı görevleri kesmedi. Havuzun tek farkı, thread referanslarıyla uğraşmaya gerek bırakmamasıdır.

12. run Metodunda Exception

`run` metodunun imzası değiştirilemez: `void run()`. Bu yüzden `run` metoduna checked exception için `throws` eklenemez. Bu javac ile denendi: `public void run() throws Exception` yazılınca `run() in Bad cannot implement run() in Runnable` hatası alındı.

Çözüm olarak hoca şunu anlattı: Multithread görevin işini, `throws` ile checked exception bildiren normal bir üye metoda yazarız. `run` metodu bu metodu `try-catch` içinde düzgün biçimde çağırır. Aşağıdaki kod bu anlatımdan kuruldu ve javac ile denendi; çıktı `Yakalandı: görev başarısız` oldu.

```
class SomeCheckedException extends Exception {
    public SomeCheckedException(String message) {
        super(message);
    }
}

public class MultiThreadedTask implements Runnable {
    private void doWork() throws SomeCheckedException {
        throw new SomeCheckedException("görev başarısız");
    }

    @Override
    public void run() {
        try {
            doWork();
        } catch (SomeCheckedException e) {
```

```
            System.out.println("Yakalandı: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        new Thread(new MultiThreadedTask()).start();
    }
}
```

Ek bilgi (derste anlatılmadı): Görevin sonuç döndürmesi ya da checked exception'ı çağırana iletmesi gerekiyorsa `Callable<V>` arayüzü ve `ExecutorService.submit` kullanılabilir. `call` metodu `throws Exception` bildirebilir. Bu javac ile denendi: `call` içinde fırlatılan exception, `Future.get` çağrısında `ExecutionException` içinde alındı.

13. Yarış Durumu ve Klasik Senkronizasyon Problemleri

Hoca "global değişken kullanmayın" uyarısının asıl konusuna böyle geldi. **Yarış durumu (race condition)**, iki ya da daha fazla thread'in ortak bir veriye erişmek istemesi ve sonucun erişim sırasına göre değişmesidir. Slaytta thread'lerin birbirinin ayağına basması ("can step on each other's toes") olarak geçer ve bozuk (corrupted) nesneler oluşabilir.

Hocanın günlük hayat örneği ortak banka hesabıdır. Bir hesabın iki kartı var, birini ailenize vermişsiniz. Hesap tek, kullanan iki kişi. Kullanıcı 1 hesaba eriştiğinde kullanıcı 2'nin o anda erişmemesi gerekir.

13.1 Neden Tek Satır Yetmez?

Para yatırma Java'da tek satır görünür:

```
balance = balance + amount;
```

İşlemci tarafında ise birden fazla komuttur. Hoca bunu Assembly mantığıyla yazdı:

```
MOV R0, balance    ; balance değerini R0'a al
ADD R0, A          ; R0'a amount kadar ekle
MOV balance, R0    ; sonucu balance'a geri yaz
```

Bir thread bu komutlardan birini çalıştırdıktan sonra sıra diğer thread'e geçebilir. Yatırma işlemi tamamlanmadan öteki thread aynı veriye erişip başka bir işlem yapabilir. Sonuçta para çektiğiniz halde çekilmemiş ya da yatırdığınız halde yatırılmamış görünebilir. Dolayısıyla paylaşılan bir değişkeni elde eden thread, işi bitip bırakana kadar oradan çıkmamalıdır.

13.2 Yemek Yiyen Filozoflar

Slaytta dört filozof (dining philosophers), dört çatal ve spagetti vardır. Bir filozofun yiyebilmesi için iki yanındaki çatalı da alması gerekir. 2 numaralı filozof iki çatalı aldıysa 1 ve 3 yiyemez, ama karşısındaki 4 yiyebilir. Yani aynı anda ancak 2 ile 4 ya da 1 ile 3 yiyebilir. 4 ve 2 çatallarını bırakırsa 1 ve 3 yiyebilir. Burada 4 ortak değişken vardır: çatallar.

13.3 Üretici ve Tüketici

Üretici-tüketici (producer-consumer) problemi, bir şeyin üretilip tüketilmesiyle ilgilidir. Hocanın örneği araç kiralama ya da kitap satın alma sayfasıdır: Bir şeyi alabilmeniz için onun önceden üretilmiş olması gerekir. Üretici, sınırlı boyuttaki bir tampona (buffer) nesne koyar: 0'dan $n-1$'e kadar n yer vardır. Tüketici buradan nesne alır. Yapı bir kuyruk gibi düşünülebilir.

- Üreticinin ekleyebilmesi için tamponda yer olmalıdır.
- Tüketicinin alabilmesi için tamponda veri olmalıdır.
- Kaynak boşken tüketici tüketemez, kaynak doluyken üretici üretemez.

Bu yüzden kaynağa erişim kontrollü yapılmalıdır.

Ders sonrası not: Altyazıda “kaynak doluyken prodüsür tüketemez” geçiyor. Üretici tüketmez, üretir; hoca büyük olasılıkla “üretemez” demek istemiştir. Yukarıda düzeltilmiş hali yazıldı.

13.4 Tek Şeritli Köprü

Bir yol bir yerde daralıp tek şeride düşer, sonra yeniden açılır. İki yönden de trafik vardır. Tek şeritli kısım paylaşılan bir değişken gibidir ve paylaşım yarış durumuna girmeyecek biçimde yapılmalıdır.

13.5 Kilitlenme ve Açlık

Paylaşımı düzenleyeyim derken yeni sorunlar çıkabilir. Hocanın benzetmesi, trafiği yönetsin diye konulan polistir.

- **Kilitlenme (deadlock):** Polis trafiği yönetirken kilitler. Thread’ler de paylaşım yaparken kilitlenebilir.
- **Açlık (starvation):** Polis bir taraftakilere hiç hak vermez, onlar bekler de bekler. Thread’e hiç sıra gelmemesidir.

Hocanın işletim sistemleri kitabından aktardığı açlık anekdotu şöyledir: 1980’li yıllarda proseslere öncelik verilen bir bilgisayarda çalışmalar bitip bilgisayar kapatılmak istenir. Sistem “çalışan bir program var, emin misiniz?” diye sorar. Bakılır ki bütün prosesler bitmiş görünürken bir prosese hiç sıra gelmemiştir, yani açlıktan ölmüştür.

14. Banka Hesabı Simülasyonu

Slaytta iki thread türü, bir hesap ve bir simülasyon sınıfı vardır. `Account` sınıfının bakiye alanı, kurucusu ve `getBalance` metodu vardır, `setBalance` yoktur. Hoca para çekme ve yatırmanın anında güncellenmediğini, araya bir gecikme konduğunu anlattı. Depositor ve Withdrawer sınıfları `Runnable`’dır, bir hesap referansı ve miktar tutar, `run` içinde sırasıyla `deposit` ve `withdraw` çağırır. Aşağıdaki kod anlatımdan kuruldu ve javac ile denendi.

```
public class Account {
    private double balance;

    public Account(double balance) {
        this.balance = balance;
    }

    public double getBalance() {
        return balance;
    }

    public void withdraw(double amount) {
        double newBalance = balance - amount;
        delay();
        balance = newBalance;
    }

    public void deposit(double amount) {
        double newBalance = balance + amount;
        delay();
        balance = newBalance;
    }

    private void delay() {
        try {
            Thread.sleep((int) (Math.random() * 5));
        } catch (InterruptedException e) {
            e.printStackTrace();
```

```
        }
    }
}

class AccountDepositor implements Runnable {
    private final Account account;
    private final double amount;

    public AccountDepositor(Account account, double amount) {
        this.account = account;
        this.amount = amount;
    }

    @Override
    public void run() {
        account.deposit(amount);
    }
}

class AccountWithdrawer implements Runnable {
    private final Account account;
    private final double amount;

    public AccountWithdrawer(Account account, double amount) {
        this.account = account;
        this.amount = amount;
    }

    @Override
    public void run() {
        account.withdraw(amount);
    }
}
```

`deposit` içindeki ilk satır çalıştıktan sonra ikinci satır çalışmadan sıra `withdraw`'a geçebilir. `withdraw` bakiyeyi okur, işini yapar, sonra `deposit` kendi eski değeriyle devam eder. Hesap karışmış olur.

Simülasyon, 0 bakiyeli bir hesap oluşturur, bir thread havuzunda 100 kez birer birim yatırır ve 50 kez birer birim çeker:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class BankSimulation {
    public static void main(String[] args) {
        Account account = new Account(0);
        System.out.println("Before: " + account.getBalance());

        ExecutorService pool = Executors.newCachedThreadPool();
        for (int i = 0; i < 100; i++) {
            pool.execute(new AccountDepositor(account, 1));
        }
        for (int i = 0; i < 50; i++) {
            pool.execute(new AccountWithdrawer(account, 1));
        }
        pool.shutdown();

        while (!pool.isTerminated()) { }
        System.out.println("After: " + account.getBalance());
    }
```

```
}
```

Beklenen sonuç: $0 + 100 \times 1 - 50 \times 1 = 50$.

Derste görülen: Before 0, After sırasıyla 6, 6, 4 ve 8 oldu. Hiçbirinde 50'ye ulaşılamadı.

Yukarıdaki yeniden kurulan kod javac ile denendi. On çalıştırmada After değerleri -1, 1, 9, 6, -2, 5, 3, 2, 2 ve -1 oldu. Sonuç, dersteki gibi 50'nin çok altında ve her seferinde farklıdır. Sebep şudur: Bir thread eski bakiyeyi okuyup gecikme boyunca beklerken öteki thread'lerin yazdığı değerler, onun geri yazmasıyla silinir. Gecikme yalnız `withdraw` içinde bırakılınca sekiz çalıştırmada 88 ile 97 arası sonuçlar çıktı. Bu yüzden kodda gecikme iki metotta da tutuldu.

> **Ders sonrası not:** `while (!pool.isTerminated()) { }` satırı, hocanın az önce kaçınılmasını söylediği busy waiting'in kendisidir; hoca burada yalnız "bitti mi bitmedi mi kontrol ediyor" dedi. Bunun yerine `pool.awaitTermination(1, TimeUnit.MINUTES)` kullanılabilir. Bu javac ile denendi: Çağrı görev bitene kadar bekledi, `true` döndürdü ve ardından `isTerminated` `true` oldu.

15. Preemptive ve Non-Preemptive Zamanlama

Hoca çözüme geçmeden önce zamanlamada (scheduling) kullanılan iki işletim sistemi kavramını andı. Google Translate'te "preemptive" kelimesi "önleyici" diye çıktı. Hoca sonra, bir prosesin belli süre (örneğin 5 birim) çalışıp bitmediyse yeniden çalışması ve bitince sıranın P_2'ye geçmesi gibi bir akış anlattı, ardından "böyle bir şey yok" diyerek konuyu bıraktı ve asıl istenenin, kaynağı tutan thread'in kritik bölgeden çıkana kadar orada kalması olduğunu söyledi.

> **Ders sonrası not:** Hoca preemptive için "hesabı elde edince bırakana kadar başkasına izin vermiyor", non-preemptive için "başkası gelebilir" dedi. Tanımlar büyük olasılıkla ters söylenmiştir. Preemptive (kesintili) zamanlamada işletim sistemi çalışan prosesi süresi dolunca ya da daha öncelikli bir proses gelince işlemciden çıkarabilir. Non-preemptive (kesintisiz) zamanlamada proses işlemciyi bitene ya da bloklanana kadar kendisi bırakmaz. Ayrıca bu kavramlar büyük olasılıkla işlemcinin paylaşımıyla ilgilidir; bir hesaba erişimi düzenleyen kavram karşılıklı dışlamadır.

16. Alice ve Bob: Kaybolan Güncelleme

Slayttaki örnekte aynı hesaba erişebilen Alice ve Bob vardır. Hesapta 1000 lira var. Hoca önce örneği bir yatırma ve bir çekme gibi okuyup 1100 ve 1050 dedi, sonra slayta bakıp düzeltti: İkisi de para çekiyor. Alice 100, Bob 50 lira çekiyor. Beklenen sonuç:

$$1000 - 100 - 50 = 850$$

Thread'ler çalıştırılınca adımlar şöyle ilerler:

Adım	Olay	Alice'in kopyası	Bob'un kopyası	Hesap
1	Alice bakiyeyi kendi alanına kopyalar	1000	-	1000
2	Bob bakiyeyi kendi alanına kopyalar	1000	1000	1000
3	Alice 100 düşer	900	1000	1000
4	Alice 900'ü hesaba yazar	900	1000	900
5	Bob kendi kopyasından 50 düşer	900	950	900
6	Bob 950'yi hesaba yazar	900	950	950

Son durumda bankada 950 lira var, Alice'in cebinde 100, Bob'un cebinde 50. Hesap sahipleri $950 + 100 + 50 = 1100$ liraya ulaşmış, yani başlangıçtaki 1000 liraya göre 100 lira "kâra geçmiştir" (python ile hesaplandı). Alice'in çekimi kaybolmuştur.

> **Ders sonrası not:** Hoca, kopyaların thread kontrol bloğuna yazıldığını söyledi. Slaydın anlattığı kopya, metodun yerel değişkenidir (`newBalance`); yerel değişkenler büyük olasılıkla thread'in kendi yığınındadır (stack), TCB'nin kendisinde değildir. Mantık değişmez: Her thread kendi kopyasıyla çalışır.

Sonuç, thread'lerin hesaba erişimini dışlamak gerektiğidir: "Sen bekle, biz işimizi bitirelim, sonra sen gireceksin." İşi bitirmek ise yerel kopyayı güncellemek değil, sonucu hesaba yazmaktır.

17. Karşılıklı Dışlama: Lock, Semafor ve Monitör

Karşılıklı dışlama (mutual exclusion) için birçok yöntem vardır.

17.1 Lock Değişkeni ve Mikrofon Mantığı

Mikrofon mantığı bu yöntemlerden biridir: Bir `lock` değişkeni olur ve onu o anda kim tutuyorsa girme hakkı onundur. Hocanın önerdiği yapı, hesap sınıfına static bir `lock` alanı eklemektir. Varsayılan değeri `false`'tur. Alice girerken `true` yapar. Bu, tuvalet kapısındaki "meşgul" yazısı gibidir. Bob girmek istediğinde `true` gördüğü için bekler. Alice çıkarken yeniden `false` yapar. Değişken static olduğu için, kaç nesne üretilirse üretilsin hepsi bu değeri görür.

> **Ders sonrası not:** Hoca değişkenin görülebilmesi için static olmasını gerekçe gösterdi. Alice ve Bob aynı hesap nesnesini paylaştığı için static olmayan bir alan da ikisi tarafından görülür; bir thread'in aynı nesnenin alanına yazdığı değerin diğer thread'den okunduğu javac ile denendi (18. bölümdeki deneme). Static yapmak ise büyük olasılıkla bütün hesapları tek kilide bağlar, yani farklı hesaplar da birbirini bekler.

Değişken `true` ve `false` yerine birkaç durumlu da olabilir: Girince 1, çıkınca 2 gibi. Üretici-tüketici problemi de bu mantıkla çözülür: Üretici sayacı 1, 2, 3, 4 diye artırır, tüketici 4, 3, 2, 1 diye azaltır. Sıfıra ya da doyum noktasına gelince durulur.

17.2 Lock Neden Tam Çözüm Değil?

Lock kısmi bir çözümdür, çünkü lock değişkeninin okunup değiştirilmesi de ayrı adımlardır. Aynı bakiyede olduğu gibi:

1. Thread A hesaba girmeden önce `lock`'u kendi hafızasına kopyalar ve `false` görür.
2. Tam o sırada Thread B de `lock`'u kopyalar ve o da `false` görür.
3. A, `lock`'u `true` yapıp girer.
4. B de elindeki `false` değerine güvenerek girer.

İki thread birlikte içeri girmiş olur. Hocanın benzetmesiyle, mikrofon bir anda klonlanmış gibidir. Dolayısıyla arada yine senkronizasyon sorunu kalır.

17.3 Semafor ve Monitör

Tam çözümlerin ayrıntısı işletim sistemleri dersinde anlatılacaktır. Biri semafordur (demir yollarındaki semaforu düşünebiliriz). Diğeri monitördür. Java'nın `synchronized` anahtar kelimesi bir monitör mekanizmasıdır.

18. synchronized ile Çözüm

Metodun başına `synchronized` yazılır. `Account` sınıfında yalnız şu üç başlık değişir:

```
public synchronized double getBalance() { ... }
public synchronized void withdraw(double amount) { ... }
public synchronized void deposit(double amount) { ... }
```

Hoca aynı simülasyonu `deposit` ve `withdraw`'a `synchronized` yazıp yeniden çalıştırdı ve Before 0, After 50 gördü. Aynı değişiklik javac ile denendi: Sonuç denenen 11 çalıştırmanın hepsinde `After: 50.0` oldu, süresi ölçülen çalıştırmalar yaklaşık 0,36 saniye sürdü.

> **Ders sonrası not:** Hoca "metodun kullandığı bütün kaynaklar bloke ediliyor, başka bir thread'in oraya girmesine izin verilmiyor" dedi. Doğrusu, `synchronized` metot o nesnenin kilidini alır ve yalnız aynı nesnedeki diğer `synchronized` kod beklemeye geçer. Bu javac ile denendi: Bir thread `synchronized` metotta 1 saniye uyurken, senkron olmayan bir metot aynı alanı 99 yapabildi; diğer `synchronized` metot ise ilkinin

çıkmasını bekledi. Bu nedenle alana erişen bütün metotların `synchronized` olması gerekir; slayttaki `getBalance` da bu yüzden işaretlidir.

18.1 Kilit Kaldırılınca Alice ve Bob

Hoca sonra Alice ve Bob örneğinde `synchronized` kaldırılırsa neler görülebileceğini tartıştı ve 950, 900 ya da 850 dedi. Her thread'in bir okuma ve bir yazma adımı olduğunu varsayarak bütün sıralamalar python ile hesaplandı:

Sıralama	Sonuç	Açıklama
Alice oku, Alice yaz, Bob oku, Bob yaz	850	İşlemler karışmadı
Bob oku, Bob yaz, Alice oku, Alice yaz	850	İşlemler karışmadı
Alice oku, Bob oku, Alice yaz, Bob yaz	950	Son yazan Bob, Alice'in çekimi kayboldu
Bob oku, Alice oku, Alice yaz, Bob yaz	950	Son yazan Bob, Alice'in çekimi kayboldu
Alice oku, Bob oku, Bob yaz, Alice yaz	900	Son yazan Alice, Bob'un çekimi kayboldu
Bob oku, Alice oku, Bob yaz, Alice yaz	900	Son yazan Alice, Bob'un çekimi kayboldu

> **Ders sonrası not:** Hoca "şu önce yazarsa 900, şu önce yazarsa 850 görürüz" dedi. Tablodan görüldüğü gibi, iki thread de eski değeri okuduysa belirleyici olan son yazandır ve sonuç 900 ya da 950 olur. 850 ise ancak işlemler birbirine karışmazsa çıkar.

Derste ekranda art arda 1950 görüldü, bir kez de 1900 yakalandı. Hoca, işlemler karışmazsa doğru sonucun da görülebileceğini söyledi ve bir ara görüldüğü söylendi; kendisi görmediğini ama görülmesinin mümkün olduğunu belirtti. Hatalı sonucu görmenin bu kadar olası olmasını, işlemlerin çok hızlı yapılması ve gecikme ile metot çağrıları sırasında sıranın başka thread'e geçmesiyle açıkladı.

Hoca, `synchronized`'ın yalnız metotta değil blok içinde de kullanıldığını, bunu slaytlarda olmasa da gelecek hafta inner class mantığıyla göstereceğini söyledi.

19. Thread-Safe Veri Yapıları

Veri yapıları konusunda kısaca bahsedilen thread-safe, yani senkronize çalışabilen yapılar burada önem kazanır. Hoca `Vector` ve `Hashtable`'ı multithreading ile kullanılabilecek yapılar olarak andı: Üzerinde thread işlemleri yapıldığında senkronize kullanıma imkan verirler.

> **Ek bilgi (derste anlatılmadı):** `Vector` ve `Hashtable` eski sınıflardır. Bugün büyük olasılıkla `ConcurrentHashMap`, `CopyOnWriteArrayList` ya da `Collections.synchronizedList` gibi seçenekler daha çok tercih edilir.

Böylece dersin slaytlardaki müfredatı bitmiştir.

20. Grafik Arayüzden Web'e: MVC'ye İlk Bakış

Hoca müfredata ek olarak thread ve GUI konusunda anlatmak istediklerini belirtti: Bunlar işletim sistemi dersinde ve web ya da mobil alanında çalışmak isteyenlere faydalı olacak. Geçen hafta C# ile başlanan görsel programlamanın Java karşılığına ve XML temelli, özellikle HTML tarafındaki yapıya bakılacak. Bunun için Visual Studio'da bir ASP.NET MVC projesi açıp kısa bir giriş yaptı.

Bir web sayfası açtığımızda gördüğümüz butonlar ve yazılar birer nesnedir, nesne tabanlı mantık burada da karşımıza çıkar. Proje açılınca bir başlangıç sınıfı ve `main` metodu vardır. Bütün alt sayfalar ve yönlendirmeler buradan başlar. Hoca bunu proje şablonunda "application start"

dosyasıyla gösterdi. Yanlışlıkla Visual Basic şablonu açtığını, C# ile çok benzer olduğunu söyledi.

MVC, projeyi üç parçaya böler. Gerçek hayattaki projelerde bir grafik arayüz, bir veri tabanı ve bir programlama tarafı vardır; programlama da iş mantığı (business logic) ve nesneye yönelik tasarım diye ikiye ayrılır.

- **View:** Web ile ilgili kısım. Derste `Home` altında `About`, `Contact` ve `Index` sayfaları görüldü. `vbhtml` dosyası biraz Visual Basic, biraz HTML içerir; C# karşılığında biraz C#, biraz HTML vardır.
- **Controller:** Görüntüyü kontrol eder. Örneğin `Index` sayfasını açmak için ilgili view dosyasını çağırır.
- **Model:** Kitaplar, yazarlar gibi veri tabanı kayıtları ve sorgular buradan yönetilir.

C# ve Visual Basic kodu daha çok controller ve modelde yazılır. Hoca, tarayıcıda sayfanın kaynağını ve "Inspect" görünümünü açarak HTML etiketlerini gösterdi: "Learn more" düğmesi başka bir sayfaya götüren bir kontroldür. Bu etiketler programlamayla üretilebilir. Örneğin view içinde bir `for` döngüsüyle `p` etiketleri 10 kez ya da n kez yazdırılabilir. Gerçek kullanımda veri tabanından bir kitap listesi çekilir, `LinkedList` ya da `ArrayList`'e alınır ve her kayıt `p` etiketlerine ya da düğmelere aktarılır. `class` ve `href` gibi özellikler de programla değiştirilebilir.

Aynı yapı başka çatılarda da vardır: PHP için CodeIgniter, Java için Spring.

Soru: MVC'nin yanında başka alanlarda kullanılan MVP ve MVVM desenlerinden de bahsedilebilir mi? **Cevap:** Olur, güzel olur. Hoca, soruyu soran öğrencinin önce genel kavramı, sonra küçük bir örneği hazırlayıp paylaşabileceğini söyledi.

Soru: Bu örnek hangi dilde olmalı? **Cevap:** Dil fark etmez. Ders nesneye yönelik programlama olduğu için Java ya da Swift gibi istenen bir dil olabilir.

Dersin Toparlaması

Bu derste önce işletim sistemi tarafındaki kavram haritası kuruldu: Program belleğe yüklenince proses olur, prosesin içindeki akışlar thread'lerdir. İşlemci birim zamanda bir komut işletir, bu yüzden paralellik ya gerçekten birden fazla çekirdekle ya da bağlam değiştirme ile sağlanan sanal paralellikle elde edilir. Sonra Java tarafına geçildi: `Runnable`, `start`, `join`, `sleep`, öncelik ve thread havuzu. Araya projede kullanılacak JUnit birim testi girdi.

Dersin ikinci yarısının ana fikri, ortak veriye erişen thread'lerin yanlış sonuç üretebilmesidir. 100 yatırma ve 50 çekmede beklenen 50 yerine 4, 6, 8 gibi değerler görüldü. Alice ve Bob örneği bunun sebebini, yani bir işlemin okuma, hesaplama ve yazma adımlarına bölünmesini gösterdi. Basit bir lock değişkeni aynı sorunu kendi içinde taşıdığı için yeterli olmadı ve `synchronized` ile sonuç 50'ye geldi. Gelecek hafta `synchronized` bloklar, GUI konusunun devamı ve C# üzerinden görsel programlama ile MVC'ye bakılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.